

Wish Mai: Play Maimai DX on iPad with Raspberry Pi Zero W and Physical Buttons

Linfan Qian

Stanford University | CS 240LX Final Project

June 9, 2026

Background and Motivation

Maimai DX is a world-famous arcade rhythm game. Its eight physical buttons provide a unique user experience, make it more engaging than touchscreen-only rhythm games.



Figure 1. maimai DX

However, many players, including myself, live far from the arcades that offer maimai DX, making it difficult to enjoy the game regularly. As a result, people seek for the solutions for playing miamai-like games at home. Fortunately, the gaming community has developed a mobile application called AstroDX, whose screen layout resembles that of maimai DX. Nevertheless, the physical buttons are the most essential component of the gameplay experience. On February 7, 2026, AstroDX v2.0.0 Release Candidate 6 introduced support for gamepad input, making it possible to use an external controller for the game. This release opened up the possibility of using a Raspberry Pi as a game controller, enabling button inputs to the Pi to be recognized by the game.

Implementation

This project consists of 2 major components: (1) DualShock 4 (DS4) controller emulation on the Raspberry Pi, including integration with button GPIO inputs, and (2) an interrupt-based asynchronous I2C implementation with an OLED display for visualizing button presses.

DS4 Emulation and Button Integration

The iPad supports USB Human Interface Devices (HIDs), including DS4. In addition, Pi Zero W's BCM2835 SoC has a DesignWare USB On-The-Go (DWC OTG) controller that can operate in USB gadget mode. Therefore, I configured the Pi to emulate a USB HID device, with iPad acting as the USB host.

Pi Zero in OTG gadget mode generally supports up to 7 active configurable endpoints, plus the non-configurable control endpoint EP0. In this project, only one endpoint, EP1 for DS4, is used.

Due to the difficulty of writing my own USB gadget from scratch, I built upon the base gadget provided by Circle, a bare-metal programming environment for the Raspberry Pi. The CDWUSBGadget class in Circle drives the hardware. I implemented a subclass CUSBDS4Gadget to provide DS4-specific descriptors and HID reports.

Circle automatically transfers the report from memory to TX via DMA when the host asks for it.

Enumeration Setup

According to Chapter 9.1.2 of `usb_20.pdf` in docs directory and online resources for USB protocol, our enumeration is divided into the following steps:

1. Host sends GET_DESCRIPTOR for device → Pi returns DS4 device descriptor
2. Host sends GET_DESCRIPTOR for configuration → Pi returns DS4 configuration descriptor, interface descriptor, HID descriptor, and endpoint descriptor (they are bundled together in this order)
3. Host sends GET_DESCRIPTOR for HID → Pi returns HID report descriptor
4. Host sends SET_CONFIGURATION → Pi activates DS4 endpoint and arms the first DMA IN (device to host) transfer
5. Host polls DS4 endpoint every 5ms → Pi responds with 64-byte DS4 input report

The DS4 descriptors are listed in detail from <https://www.psdevwiki.com/ps4/DS4-USB>.

Feature Report

Before fully accepting the device as a DS4 controller, the host issues GET_REPORT to request some device features, including pairing information, firmware version, and IMU calibration data. Circle forwards those to subclasses to handle through `CSWUSBGadget::OnClassOrVendorRequest()`.

Input Report

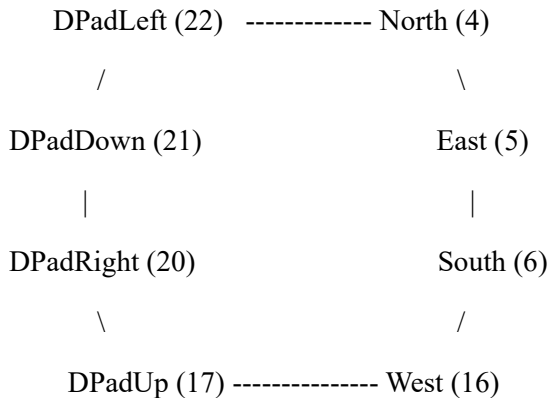
The DS4 input report is 64 bytes long and is transmitted from the device to the host. It encodes the state of each controller component, including but not limited to face buttons, D-pad, and

joysticks. The full structure can be found from [dualshock 4 input report usb](#) and [dualshock 4 input report common](#) in linux/drivers/hid/hid-playstation.c.

Button Integration

Button inputs are read through GPIO polling rather than interrupts. This approach allows multiple button presses to be captured simultaneously and included in a single input report sent to the host. In contrast, an interrupt-based design would process button presses individually, potentially resulting in multiple reports being sent and increased latency for buttons that were pressed at nearly the same time.

The diagram below illustrates the physical button layout and corresponding GPIO mappings.



Dpad hat encodes 8 cardinal and diagonal directions instead of using one bit for each button. Therefore, Up+down or left+right are physically impossible. When such conflicts occur, I break them into 2 separate reports.

Interrupt-Based I2C and OLED

I used the SSD1306 OLED to display the real-time button presses. See [CS 240LX OLED lab](#) for more SSD1306-specific information.

The write provided by Circle built-in I2C master is in the blocking mode. However, minimizing delay is crucial for a rhythm game, so we don't want the writes to OLED block our button reads. As a result, I implemented an interrupt-based I2C that enables the asynchronous I2C write.

The BCM2835 I2C control register and status register support interrupts on TX. That enables us to implement the asynchronous write as the following:

1. When WriteAsync() is called, load leading bytes to FIFO and enables the TX interrupt.
2. In the interrupt handler, continue filling FIFO with remaining data if there's any. If the transfer is done, flag the "done" status and disable the interrupt.

WriteAync() requires the buffer pointer passed to be persistent so that it can reduce one memcpy to save time.

Resources

AstroDX: <https://github.com/2394425147/astrodx>

Circle: <https://github.com/rsta2/circle>

DS4 wiki: <https://www.psdevwiki.com/ps4/DS4-USB>

Linux USB code for DS4: <https://github.com/torvalds/linux/blob/master/drivers/hid/hid-playstation.c>

Stanford CS 240LX course site: <https://github.com/ddrrreee/cs240lx-26spr>